

Что сделано в лабораторной работе

Общая идея

Была разработана веб-система «Книжный каталог» — сайт для библиотеки, где читатели могут искать и бронировать книги, а библиотекарь управляет каталогом. Система написана на ASP.NET WebForms (C#) и хранит все данные в базе данных MS SQL Server.

До этой лабораторной работы данные хранились прямо в памяти программы — то есть при каждой перезагрузке сервера всё терялось. В этой работе была подключена настоящая база данных, чтобы данные сохранялись постоянно.

Как устроена база данных

База данных называется `BookCatalogDB`. В ней четыре таблицы:

`Users` — хранит всех пользователей системы. У каждого есть имя, email, пароль, роль (читатель или администратор) и лимит бронеи.

`Books` — каталог книг. Для каждой книги хранится название, автор, ISBN, жанр, год, количество всего экземпляров и сколько из них сейчас свободно. Есть флаг `IsArchived` — если книга списана, она скрывается из каталога но не удаляется.

`BookReservations` — все бронирования. Связывает пользователя с книгой, хранит дату начала и конца брони, статус (активна, истекла, отменена, в очереди).

`WriteOffRecords` — архив списаний. Когда книгу списывают, сюда сохраняется запись с причиной и «снимком» данных книги в формате JSON — чтобы можно было восстановить в течение 24 часов.

Как создавалась база данных

В SQL Server Management Studio (SSMS) был выполнен SQL-скрипт, который создал базу данных и все таблицы с нужными ограничениями — например,

email должен быть уникальным, год книги должен быть от 1000 до 2026, статус брони может быть только одним из допустимых значений.

Были созданы индексы — специальные структуры, которые ускоряют поиск. Например, индекс по полям `Title`, `Author`, `Genre`, `Year` позволяет быстро фильтровать книги.

Были созданы хранимые процедуры — это готовые SQL-команды, которые хранятся прямо на сервере БД. Они быстрее работают, чем если бы SQL писался каждый раз в коде. Процедуры:

- `sp_RegisterUser` — регистрирует нового пользователя
- `sp_AuthenticateUser` — проверяет логин и пароль
- `sp_SearchBooks` — ищет книги по фильтрам
- `sp_CreateBooking` — создаёт бронь
- `sp_AddBook` — добавляет книгу или увеличивает количество экземпляров
- `sp_WriteOffBook` — списывает книгу

В файле `Web.config` была прописана строка подключения — адрес сервера и название базы данных, чтобы сайт знал куда подключаться.

Как сайт работает с базой данных

Для работы с базой данных написаны два вспомогательных класса:

`DBHelper.cs` — низкоуровневый класс. Он умеет открывать соединение с базой, выполнять хранимые процедуры или SQL-запросы и возвращать результат. Всё соединение открывается и закрывается автоматически через `using` — утечек памяти нет.

`DataStore.cs` — класс с методами для каждого действия в системе: поиск книг, регистрация, бронирование, списание и так далее. Все страницы сайта обращаются только к `DataStore`, а он уже использует `DBHelper` для работы с базой.

Как работает каждая функция

Регистрация. Пользователь вводит данные на форме. Сайт проверяет, нет ли уже такого email в таблице `Users`. Если нет — вызывается процедура `sp_RegisterUser`, которая добавляет нового пользователя. Пользователь сразу входит в систему.

Поиск книг. На главной странице пользователь вводит фильтры. Сайт вызывает `sp_SearchBooks` с переданными параметрами. Процедура ищет в таблице `Books` только неархивные книги и возвращает список. Параметры передаются безопасно через `SqlParameter` — SQL-инъекции невозможны.

Бронирование. Пользователь нажимает «Забронировать». Сайт проверяет, что книга доступна и лимит броней не превышен. Затем вызывается `sp_CreateBooking`, которая в транзакции одновременно создаёт запись в `BookReservations` и уменьшает `AvailableCount` в таблице `Books`. Транзакция нужна, чтобы не получилось ситуации когда бронь создана, а счётчик не уменьшился — или наоборот.

Добавление книги. Администратор заполняет форму. Сначала проверяется, нет ли книги с таким ISBN. Если есть — предлагается просто добавить экземпляры. Если нет — создаётся новая запись через `sp_AddBook`.

Списание. Администратор выбирает книгу и причину. Если все экземпляры свободны — книга помечается как `IsArchived = 1` и в `WriteOffRecords` сохраняется JSON-снимок с данными книги. Если есть активные брони — система предупреждает и предлагает их отменить. В течение 24 часов книгу можно восстановить из снимка.

Истечение броней. При каждой загрузке главной страницы автоматически запускается запрос, который находит все брони с истёкшей датой, переводит их в статус `Expired` и возвращает книги в «В наличии».

Главный результат

Система перешла от хранения данных в памяти к полноценной реляционной базе данных. Данные теперь сохраняются между перезапусками, защищены ограничениями на уровне SQL Server и обрабатываются транзакционно — то есть либо все изменения применяются, либо ни одно.